Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Seminario de Sistemas I.
Ing. Mario Tobar.

“DOCUMENTACIÓN - GUÍA DE DESPLIEGUE Y RUNBOOK”

Diego José Maldonado Monterroso.
Carné: 201931811.

Quetzaltenango, Guatemala.

Lunes 26 de Octubre de 2026.

Introducción

Este documento explica cómo se despliega y cómo se opera el sistema de agendamiento y expediente clínico de la clínica de psicología. Su audiencia es quien lo despliega por primera vez y quien lo mantiene en producción.

Capítulo	Responde
Guía de despliegue	Qué hay en la nube, cómo llega una versión a producción y cómo se despliega desde cero
Respaldos y recuperación	Qué se respalda, cada cuánto y en cuánto tiempo se recupera
Monitoreo	Qué se vigila, dónde, con qué umbral y cada cuánto
Reversión	Cómo volver a la versión anterior de la aplicación, la base, la configuración o la infraestructura
Gestión de incidentes	Severidades, roles, escalamiento, comunicación y post-mortem
Procedimientos	Los pasos exactos, RB-01 a RB-18
Bitácora de operación	Lo que ya se ejecutó en producción

Convenciones de los comandos:

- `<dominio>` es `pf-clinica.duckdns.org` y `<entorno>` es `dev`, `stage` o `prod`;
- `<ec2_host>` es la IP elástica de la EC2, salida de Terraform;
- los comandos `aws` corren en la máquina del operador, con la región `us-east-1`;
- los de `docker compose` corren en la EC2, dentro de `~/clinica/<entorno>`;
- a la EC2 se entra con `ssh -i ~/.ssh/clinica-ec2 ubuntu@<ec2_host>`.

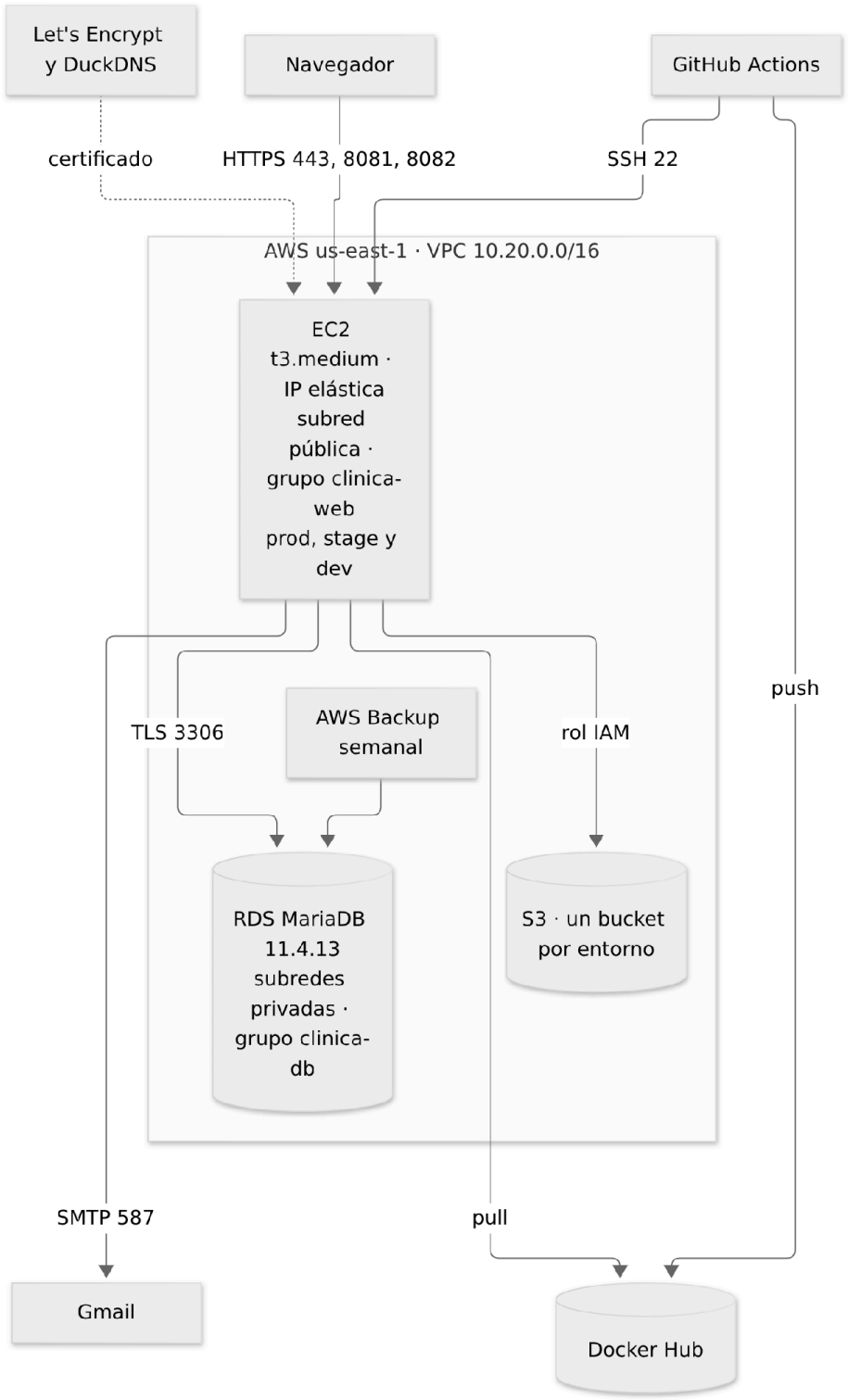
Guía de despliegue

El sistema corre en Amazon Web Services (AWS), región `us-east-1`. Terraform crea la infraestructura y GitHub Actions despliega la aplicación. Hay tres entornos, cada uno con su base de datos y su bucket:

Entorno	Dirección	Se despliega	Uso
prod	`https://pf-clinica.duckdns.org`	A mano desde `main`, con aprobación	La clínica
stage	`https://pf-clinica.duckdns.org:8081`	Solo, al fusionar un Pull Request en `main`	Validar la versión que irá a prod
dev	`https://pf-clinica.duckdns.org:8082`	Solo, con cada Pull Request abierto	Probar el cambio antes de fusionarlo

El costo es de unos USD 50 al mes. Un presupuesto de AWS avisa por correo al llegar al 80 % de USD 85.

Arquitectura desplegada



Recurso	Configuración
Red	VPC `10.20.0.0/16` con una subred pública y dos privadas en zonas distintas, sin NAT
Grupo `clinica-web`	Entrada por 22 (despliegue), 80 (redirección a HTTPS), 443, 8081 y 8082
Grupo `clinica-db`	Entrada por 3306 solo desde `clinica-web`
EC2	`t3.medium` (2 vCPU, 4 GB y 2 GB de *swap*), Ubuntu 24.04, disco de 30 GB cifrado, IMDSv2 obligatorio e IP elástica
RDS	MariaDB 11.4.13 `db.t4g.micro`, 20 GB cifrados, privada, TLS obligatorio, recuperación a un instante (PITR) de 7 días y protección contra borrado
S3	Un bucket por entorno (`clinica-<entorno>-archivos-<cuenta>`), privado y versionado
AWS Backup	Copia completa de RDS cada domingo a las 02:00 de Guatemala, guardada 4 semanas
IAM	Rol `clinica-ec2`: leer, escribir y borrar objetos solo en los tres buckets. Rol `clinica-infra`: lo asume únicamente el pipeline de infraestructura por OIDC. Rol de AWS Backup con la política administrada del servicio
Estado de Terraform	Bucket `clinica-tfstate-<cuenta>`, privado y versionado, con bloqueo por archivo
Presupuesto	USD 85 al mes con aviso al 80 %

Cada entorno es un proyecto de Docker Compose en la EC2, con estos contenedores:

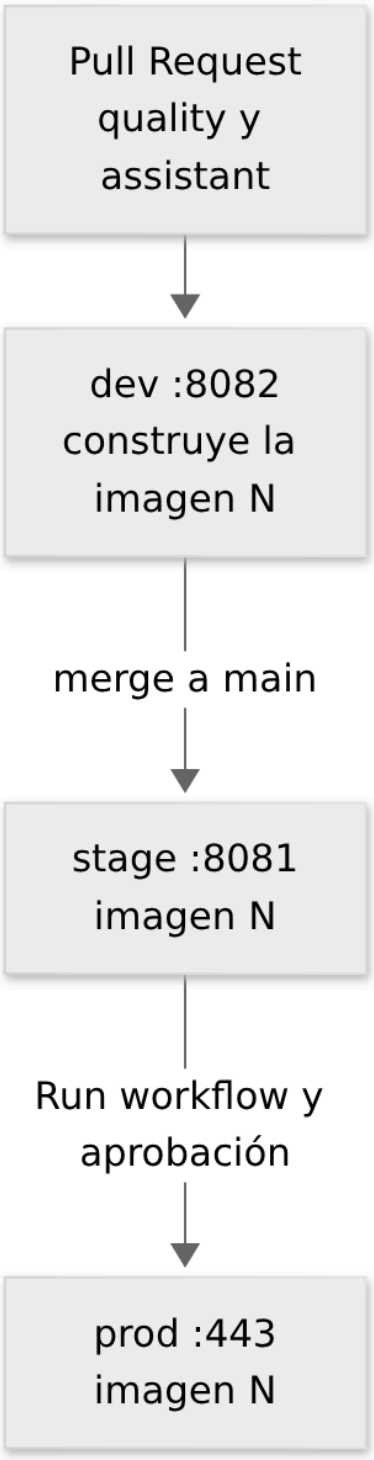
Contenedor	Qué hace
`web`	nginx 1.30.5 sin privilegios: termina TLS con HSTS, sirve los estáticos y pasa PHP a `app`
`app`	PHP-FPM 8.4 con Laravel 13, Filament 5 y Livewire 4
`worker`	Atiende las colas `critical`, `mail`, `documents` y `default`; se reinicia solo cada hora
`scheduler`	Concilia citas cada 5 minutos y vence pruebas psicométricas cada hora
`chatbot`	El asistente en Python 3.13 y FastAPI. No publica puertos: solo Laravel le habla, con token
`redis`	Solo caché. Perderlo no pierde datos: sesiones y colas viven en la base

Herramientas y versiones

Herramienta	Versión	Dónde
Terraform	1.16.6, en contenedor	Máquina del operador y pipeline `infra.yml`
Proveedor `hashicorp/aws`	6.69.1, fijado en `.terraform.lock.hcl`	`infra/`
Docker y Compose	29.9.1 y 5.6.0, instalados por `user-data.sh`	EC2

Herramienta	Versión	Dónde
GitHub Actions	Workflows `ci-cd.yml` e `infra.yml`	Repositorio `DiegoMaldonado19/clinica-app`
Registro de imágenes	Docker Hub: `clinica-app`, `clinica-web` y `clinica-chatbot`	Público; las imágenes no llevan secretos

Pipeline de integración y entrega continua



Job	Cuándo	Qué hace
`quality`	Cada Pull Request y cada push a `main`	Pint, PHPStan con Larastan, Deptrac, pruebas unitarias, migraciones sobre MariaDB, pruebas de integración y funcionales, una corrida sin caché, `composer audit` y gitleaks
`assistant`	Igual que `quality`	ruff, mypy, pytest con la batería de crisis y pip-audit del asistente
`deliver`	Si los dos anteriores pasan, o por despacho manual hacia prod	Construye las tres imágenes o reutiliza las del mismo árbol de archivos, las publica, entra por SSH, renueva TLS si hace falta, despliega y comprueba `/up` desde fuera

Una imagen por versión. Cada imagen lleva la huella del árbol de archivos. Si el árbol ya tiene imagen, se promueve esa misma: stage y prod reciben exactamente lo que se probó en dev. Prod nunca construye; toma la imagen del último despliegue sano de stage.

El despliegue en la EC2 lo hace `scripts/deploy.sh`:

1. crea la base y el usuario del entorno si no existen;
2. descarga las imágenes;
3. migra y siembra los catálogos con `php artisan migrate --force --seed`;
4. levanta los contenedores con `docker compose up --wait`;
5. borra las imágenes sin uso.

Las migraciones son versionadas y automáticas. Corren en cada despliegue y siempre hacia adelante: un cambio de esquema se corrige con una migración nueva, nunca editando una ya aplicada.

Los secretos viven en los Environments de GitHub (`dev`, `stage`, `prod` e `infra`). El pipeline escribe el `.env` de cada entorno en la EC2 y falla si falta un valor. Ninguno está en el repositorio, y gitleaks revisa cada Pull Request.

Flujo de ramas

El proyecto lo desarrolla una sola persona. Por eso usa desarrollo basado en tronco (*trunk-based*): una rama corta por cambio y un Pull Request hacia `main`. Así lo acordó

con los evaluadores desde la primera fase. Cada control de GitFlow tiene su equivalente:

Control	Cómo se cumple
Nada llega a `main` sin revisión automática	`main` exige Pull Request y los checks `quality` y `assistant` en verde, con la rama al día
Integración antes de producción	Cada Pull Request se despliega en dev; cada fusión, en stage
Producción solo con aprobación	El Environment `prod` exige la aprobación del responsable
Corrección urgente	Una rama corta desde `main` con su Pull Request: recorre dev y stage y se aprueba hacia prod
Infraestructura controlada	`infra.yml` corre solo a mano, desde `main` y con aprobación del Environment `infra`

Primer despliegue desde una cuenta vacía

Son unas dos horas, la mitad de espera.

#	Paso	Deja listo	Tiempo
1	Cuentas y llaves	AWS, SSH, DuckDNS, Gmail y Docker Hub	25 min
2	Verificaciones previas	Que el `apply` no falle a medias	5 min
3	Bootstrap	Bucket del estado y rol OIDC de GitHub	3 min
4	Infraestructura	VPC, EC2, RDS, S3, respaldos y presupuesto	20 min
5	GitHub	Variables, Environments, secretos y protección de `main`	20 min
6	dev, stage y prod	Los tres entornos con la misma imagen	35 min
7	Primer ingreso	La psicóloga con contraseña y segundo factor	5 min
8	Infraestructura por el pipeline	Un `plan` sin cambios por OIDC	5 min

1. Cuentas y llaves

Cada valor se guarda en un gestor de contraseñas: GitHub no deja volver a leer un secreto.

Dato	Cómo se obtiene
Llave de AWS	Usuario IAM `clinica-admin` con `AdministratorAccess`, solo para el bootstrap. Se desactiva al terminar
Llave SSH del pipeline	`ssh-keygen -t ed25519 -f ~/.ssh/clinica-ec2 -N '' -C clinica-pipeline`
Subdominio y token de DuckDNS	En duckdns.org. La IP la registra la EC2
Contraseña de aplicación de Gmail	Con la verificación en dos pasos de la cuenta activa; se copia sin espacios

Dato	Cómo se obtiene
Token de Docker Hub	De lectura y escritura
Datos de la clínica	Teléfono, correo, dirección y línea de crisis confirmada, que la landing muestra siempre

2. Verificaciones previas

- La cuenta de AWS debe estar en el plan de pago: el plan gratuito rechaza la `t3.medium`.
- La AMI de Ubuntu fijada en `ec2.tf` debe existir en la región.
- Si la cuenta ya tiene el proveedor OIDC de GitHub, se importa antes de aplicar el bootstrap.

Terraform corre en contenedor, con la misma versión que el pipeline:

```
alias terraform='docker run --rm -it -u "$(id -u):$(id -g)" -e HOME=/tmp -v
"$HOME/.aws:/tmp/.aws:ro" \
  -e AWS_PROFILE -e TF_VAR_db_admin_password -e TF_VAR_ssh_public_key -e
TF_VAR_alert_email \
  -v "$PWD:/w" -w /w hashicorp/terraform:1.16.6'
```

3. Bootstrap, una sola vez

```
cd infra/bootstrap
terraform init && terraform apply      # anota state_bucket e infra_role_arn
```

4. Infraestructura

```
cd ..
export TF_VAR_db_admin_password="$(openssl rand -hex 20)"   # es
DB_ADMIN_PASSWORD
export TF_VAR_ssh_public_key="$(cat ~/.ssh/clinica-ec2.pub)"
export TF_VAR_alert_email="correo-del-responsable"
terraform init -backend-config="bucket=<state_bucket>"
terraform apply                         # anota ec2_host y db_host; RDS tarda
unos 15 minutos
```

La contraseña maestra de RDS es de solo escritura: no queda en el estado de Terraform.

5. GitHub

Variables del repositorio: `DOMAIN`, `CLINIC_PHONE`, `CLINIC_EMAIL`, `CLINIC_ADDRESS` y `CLINIC_CRISIS_LINE`. Son datos públicos y van como variables, no como secretos.

Environments:

Environment	Ramas permitidas	Aprobación
`dev`	Cualquiera (corre sobre el Pull Request)	No
`stage`	`main`	No
`prod`	`main`	Sí, del responsable
`infra`	`main`	Sí, del responsable

Secretos de dev, stage y prod. «Igual» lleva el mismo valor en los tres entornos y «distinto» se genera aparte para cada uno.

Secreto	Valor
`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`	Igual: la cuenta de Docker Hub
`EC2_HOST`, `EC2_SSH_KEY`	Igual: la salida `ec2_host` y la llave privada del pipeline
`DB_HOST`, `DB_ADMIN_PASSWORD`	Igual: la salida `db_host` y la contraseña maestra
`DB_PASSWORD`	Distinto: `openssl rand -hex 24`
`APP_KEY`, `CLINICAL_ENCRYPTION_KEY`	Distintos entre sí y por entorno: `base64:` más `openssl rand -base64 32`. Perder la llave clínica de prod es perder las notas
`CHATBOT_API_TOKEN`	Distinto: `openssl rand -hex 32`
`AWS_BUCKET`	Distinto: `clinica-<entorno>-archivos-<cuenta>`
`MAIL_USERNAME`, `MAIL_PASSWORD`	Igual: la cuenta de Gmail, que también es el remitente, y su contraseña de aplicación
`ADMIN_EMAIL`	Igual: el correo real de la psicóloga, con el que entra la primera vez
`DUCKDNS_TOKEN`, `TLS_EMAIL`	Igual: el token de DuckDNS y el correo de avisos de Let's Encrypt
`ANTHROPIC_API_KEY`, `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`	Opcionales. Sin la primera, el asistente se despliega apagado; sin las otras dos, todo aviso sale por correo

Secretos de `infra`: `AWS_ROLE_ARN` (la salida `infra_role_arn`), `TF_STATE_BUCKET`, `TF_VAR_db_admin_password`, `TF_VAR_ssh_public_key` y `TF_VAR_alert_email`.

Protección de `main`: Pull Request obligatorio sin aprobaciones (el único desarrollador no puede aprobarse a sí mismo), checks `quality` y `assistant` obligatorios y rama al día.

6. dev, stage y prod

1. **dev:** se abre el Pull Request. `deliver` construye las imágenes y despliega. Debe responder `https://<dominio>:8082/up`.

2. **stage:** al fusionar, se despliega la misma imagen en `https://<dominio>:8081`.
3. **prod:** Actions → CI/CD → *Run workflow* desde `main` → *Review deployments* → *Approve*.

El primer despliegue de cada entorno espera a que la EC2 termine de instalar Docker, emite el certificado de Let's Encrypt (uno para los tres puertos), crea la base y siembra los catálogos y la clínica.

7. Primer ingreso

La psicóloga entra a `https://<dominio>/admin` por «¿Olvidaste tu contraseña?» con el correo de `ADMIN_EMAIL`. Elige su contraseña, configura el segundo factor, guarda sus códigos de recuperación y crea desde el panel el usuario de recepción.

8. Infraestructura por el pipeline

Actions → Infraestructura → *Run workflow* → `plan` → aprobación. Debe terminar en `No changes`: prueba que GitHub entra a AWS por OIDC, sin llaves guardadas. Después se desactiva la llave de `clinica-admin`.

Despliegue de una versión nueva

1. Abrir el Pull Request y esperar `quality`, `assistant` y el despliegue en dev.
2. Probar el cambio en `https://<dominio>:8082`.
3. Fusionar. Esperar el despliegue en stage y probar en `https://<dominio>:8081`.
4. Actions → CI/CD → *Run workflow* desde `main` y aprobar `prod`.
5. Verificar prod con la lista de abajo.

Verificación después de desplegar

Comprobación	Cómo	Resultado esperado
Salud	`curl -s -o /dev/null -w '%{http_code}' https://<dominio>/up`	`200` en los tres puertos
Redirección	`curl -sI http://<dominio>/`	`301` hacia `https://`
Certificado	`openssl s_client -connect <dominio>:443 -servername`	Emitido por Let's Encrypt y vigente

Comprobación	Cómo	Resultado esperado
	`<dominio> </dev/null | openssl x509 -noout -issuer -dates`	
Imagen	En la EC2: `cd ~/clinica/prod && docker compose ps`	El mismo número de imagen que stage, contenedores sanos
Ingreso	`https://<dominio>/admin` y `/portal`	La pantalla de ingreso, sin errores

Si algo falla

Síntoma	Causa probable	Qué hacer
«Escribir el .env» falla por valores faltantes	Una variable guardada como secreto, o un secreto vacío	Mover `DOMAIN` y `CLINIC_*` a variables y completar lo que indica el registro
`Could not assume role with OIDC`	`AWS_ROLE_ARN` distinto del rol creado, o la confianza no coincide con el repositorio	Comparar con `terraform output -raw infra_role_arn` en `infra/bootstrap`
`docker/login-action` falla	Token de Docker Hub vencido	Crear otro de lectura y escritura
No conecta por SSH	`EC2_HOST` equivocado, o la EC2 aún arranca	Revisar `terraform output ec2_host` y repetir
`DuckDNS rechazo la actualizacion: KO`	Token o subdominio	Copiar de nuevo el token; `DOMAIN` es `<subdominio>.duckdns.org`
«Verificar desde fuera» falla	Un contenedor no quedó sano	`docker compose ps` y `docker compose logs app web` en la EC2
prod: `Stage todavia no tiene un despliegue exitoso`	stage no terminó bien	Desplegar stage primero

Límites declarados

- Los tres entornos comparten la EC2 y su rol IAM; aislarlos exige una instancia por entorno.
- El puerto 22 está abierto porque los ejecutores de GitHub no tienen IP fija. Solo se entra con la llave del pipeline.
- No hay vuelta atrás automática. Si los contenedores nuevos no quedan sanos, el entorno sigue con esa versión hasta desplegar la anterior. Si falla antes, al migrar, siguen sirviendo los contenedores anteriores.

Respaldos y recuperación

Política

La política exige tres capas: copia completa semanal, copia incremental diaria y respaldo del registro de transacciones cada 15 a 30 minutos, para recuperar a un instante concreto.

Capa exigida	Cómo se cumple	Retención
Completa semanal	AWS Backup copia la instancia RDS cada domingo a las 02:00 de Guatemala, en la bóveda `clinica`	4 semanas
Incremental diaria	RDS toma su instantánea automática diaria; desde la segunda, cada una guarda solo los bloques que cambiaron	7 días
Registro de transacciones	RDS sube el registro binario cada 5 minutos. Permite restaurar a cualquier segundo de los últimos 7 días	7 días
Archivos	Comprobantes y fotos en S3 con versionado: un borrado o una sobrescritura conserva la versión anterior	Indefinida
Llaves de cifrado	`CLINICAL_ENCRYPTION_KEY` y `APP_KEY` de prod, en el Environment `prod` y en el respaldo fuera de línea del responsable	Indefinida

Sin la llave clínica, un respaldo no sirve: las notas SOAP y las respuestas de las pruebas están cifradas por la aplicación. Toda restauración se verifica descifrando una nota (RB-03).

La EC2 no guarda estado: los datos viven en RDS y S3. Perder la instancia no pierde información.

Objetivos de recuperación

El simulacro mensual (RB-04) mide el RTO y el RPO reales.

Escenario	RTO (tiempo para volver)	RPO (datos que se pueden perder)	Procedimiento
Error lógico o datos dañados	30 a 60 minutos	5 minutos	RB-03
Falla de la instancia RDS (una sola zona)	10 a 20 minutos	5 minutos	RB-03, con el último instante restaurable
Pérdida o compromiso de la EC2	Unos 30 minutos	Ninguno	RB-06
Contenedor caído o colgado	Menos de 5 minutos	Ninguno	RB-09

Escenario	RTO (tiempo para volver)	RPO (datos que se pueden perder)	Procedimiento
Borrado de un archivo en S3	15 minutos	Ninguno	Restaurar la versión anterior del objeto

Monitoreo

Qué se vigila

Señal	Dónde se mira	Umbral	Frecuencia	Si se cruza
Disponibilidad	`curl https://<dominio>/up` en los tres puertos; el *healthcheck* de `web` cada 10 s; el paso «Verificar desde fuera» de cada despliegue	Distinto de `200`	Diaria y en cada despliegue	RB-09
Contenedores	`docker compose ps`	`unhealthy` o reinicios en bucle	Diaria	RB-09
CPU y memoria de la EC2	`free -h`, `docker stats --no-stream`; CPU en CloudWatch (métrica básica de EC2)	Memoria o CPU sobre 80 % sostenido	Semanal	RB-09; si persiste, separar prod en su propia instancia
Disco de la EC2	`df -h /` y `docker system df`	Sobre 80 %	Semanal	RB-13
Base de datos	CloudWatch de RDS: `CPUUtilization`, `FreeStorageSpace`, `DatabaseConnections`, `FreeableMemory`	CPU sobre 80 %, menos de 4 GB libres	Semanal	RB-16
Colas	Conteo de `jobs` y `failed_jobs`	Más de 50 en cola, o cualquier fallido	Diaria	RB-07
Correo	Aviso NT-20 (un envío falló 3 veces), indicador de entrega del tablero, `notification_dispatches` fallidos	Cualquier NT-20	Al llegar	RB-08
Certificado TLS	`~/clinica/tls.log` y la fecha de vencimiento con `openssl`	Menos de 20 días de vigencia	Semanal	RB-14
DNS	`dig +short <dominio>`	Distinto de la IP elástica	Si falla el acceso	RB-15
Respaldos	AWS Backup → *Jobs*; `LatestRestorableTime` de RDS	Un trabajo fallido, o el último instante restaurable con más de 15 minutos de atraso	Semanal	Revisar el trabajo y, si falla otra vez, abrir caso con AWS
Gasto	Aviso de AWS Budgets por correo	80 % de USD 85	Al llegar	Revisar el gasto por servicio en Billing
Seguridad	Bitácora del panel (lecturas del expediente, intentos denegados,	Patrón anómalo	Semanal	RB-12

Señal	Dónde se mira	Umbral	Frecuencia	Si se cruza
	códigos de segundo factor equivocados) y CloudTrail (90 días)			
Alertas clínicas	Correos NT-24 (crisis en el asistente) y NT-25 (ítem 9 del PHQ-9)	Cualquiera	Al llegar	RB-11

Dónde están los registros

Registro	Comando	Retención
Contenedores	`docker compose logs --tail 200 <servicio>`	Rotados a 5 archivos de 10 MB por contenedor
Laravel	`docker compose exec app tail -n 200 storage/logs/laravel.log`	Hasta el siguiente despliegue: para conservarlo, `docker compose cp app:/var/www/html/storage/logs/laravel.log .` antes
Renovación de TLS	`tail -n 50 ~/clinica/tls.log`	Sin rotación
Despliegues	GitHub → Actions y Deployments	Los de GitHub
Acciones en el sistema	Panel → Bitácora (tabla `audit_log`, inmutable)	Indefinida
Llamadas a la API de AWS	CloudTrail → *Event history*	90 días

Rutina

Cada	Revisión
Día	`/up` de prod; correos NT-20, NT-24 y NT-25; trabajos fallidos
Semana	Disco y memoria de la EC2; métricas de RDS; trabajo de AWS Backup del domingo; vigencia del certificado; bitácora del panel
Mes	Simulacro de restauración (RB-04); gasto del mes; alertas NT-24 para recalibrar el clasificador de crisis
Seis meses	Rotación de secretos (RB-05)

Revisión rápida de salud, desde la EC2:

```
for p in 443 8081 8082; do curl -s -o /dev/null -w "$p %{http_code}\n"
"https://<dominio>:$p/up"; done
free -h && df -h / && docker system df
cd ~/clinica/prod && docker compose ps
docker compose exec app php artisan tinker --execute 'echo
DB::table("jobs")->count()." en cola, ".DB::table("failed_jobs")->count()."
fallidos";'
```

Reversión

Qué falló	Cómo se revierte	Procedimiento
Una versión de la aplicación en dev o stage	Volver a correr la última ejecución sana: su árbol ya tiene imagen y se promueve sin construir	RB-02
Una versión en prod	`git revert` por Pull Request; stage recibe la versión anterior y se despacha prod	RB-02
Un cambio de esquema	Nunca hacia atrás: una migración nueva corrige hacia adelante	RB-02
Datos dañados	Restaurar la base a un instante anterior en una instancia aparte	RB-03
Un secreto o variable	Volver a poner el valor anterior en el Environment y repetir el despliegue	RB-05
La infraestructura	Revertir el Pull Request de `infra/` y aplicar con el pipeline de infraestructura	`plan` y luego `apply`, con aprobación
La EC2	Reemplazarla: la IP elástica pasa a la nueva	RB-06

Gestión de incidentes

Severidades

Nivel	Qué es	Ejemplos	Respuesta	Resolución objetivo
SEV-1	Sistema caído o datos clínicos expuestos	prod sin responder; base inaccesible; acceso indebido a expedientes	Inmediata	1 hora
SEV-2	Una función crítica no sirve	No se puede agendar, pagar ni entrar (segundo factor o correo caídos); certificado vencido	30 minutos	4 horas
SEV-3	Función menor con alternativa	Exportación del tablero lenta; asistente apagado; un aviso no salió	El mismo día	Siguiente día hábil
SEV-4	Detalle visual o de redacción	Un texto mal escrito	Al planificar	Próxima versión

Roles y escalamiento

Rol	Quién	Hace
Responsable técnico	El desarrollador (Diego José Maldonado Monterroso)	Detecta, clasifica, contiene, resuelve, documenta y escribe el post-mortem
Responsable de los datos	La psicóloga	Se le informa todo SEV-1 y SEV-2, y todo incidente con datos de pacientes; decide si se restaura
Recepción	Personal de la clínica	Avisa a los pacientes con citas afectadas y agenda por teléfono mientras dure el incidente

Si el problema es de	Escalar a
EC2, RDS, S3 o AWS Backup	Panel de estado de AWS (*Health Dashboard*) y caso de soporte
Despliegues o Actions	Página de estado de GitHub
Imágenes	Página de estado de Docker Hub
DNS	DuckDNS; si cae por horas, RB-15
Correo	Estado de Google Workspace y Gmail

Ciclo de un incidente

1. **Detectar:** un umbral del monitoreo, un aviso por correo o un reporte de la clínica.
2. **Clasificar** la severidad con la tabla de arriba.
3. **Contener:** evitar que empeore; en seguridad, rotar y aislar primero (RB-12).

4. **Resolver** con el procedimiento que corresponde.
5. **Verificar** con la lista de verificación posterior al despliegue.
6. **Comunicar** el cierre a la psicóloga.
7. **Documentar** en la bitácora de operación; para SEV-1 y SEV-2, escribir el post-mortem en los dos días siguientes.

Mensaje base para la clínica:

```
Asunto: [SEV-n] <servicio afectado> — <abierto | resuelto>
Desde <hora> el sistema <síntoma>. Afecta a <quién>. Mientras tanto,
<alternativa>.
Próxima actualización: <hora>. Responsable: <nombre>.
```

Plantilla de post-mortem

Campo	Contenido
Resumen	Qué pasó, en dos líneas
Severidad y duración	Nivel, inicio, detección, contención y cierre (UTC y Guatemala)
Impacto	Usuarios, citas, pagos o datos afectados
Línea de tiempo	Eventos con hora
Causa raíz	La causa, no el síntoma
Qué funcionó y qué no	Detección, procedimientos y herramientas
Acciones	Cada cambio que evita que se repita, con responsable y fecha

Sin culpas: se buscan causas en el sistema, no responsables.

Procedimientos

Código	Procedimiento	Cuándo
RB-01	Desplegar a producción	Por versión
RB-02	Revertir un despliegue	Ante una versión fallida
RB-03	Restaurar la base a un instante concreto	Datos dañados o falla de RDS
RB-04	Simulacro de restauración	Mensual
RB-05	Rotar un secreto	Ante exposición o cada seis meses
RB-06	Reemplazar la instancia	EC2 comprometida o irrecuperable
RB-07	Diagnosticar trabajos encolados detenidos	Cola creciente o trabajos fallidos
RB-08	Diagnosticar correo no entregado	NT-20 o un usuario sin códigos
RB-09	Recuperar un entorno degradado sin desplegar	Contenedores caídos o lentos
RB-10	Actualizar lo que sabe el asistente	Cambios de tarifas, servicios o preguntas
RB-11	Responder a una alerta de riesgo	NT-24 o NT-25
RB-12	Responder a un incidente de seguridad	Exposición o acceso indebido
RB-13	Liberar disco en la EC2	Disco sobre 80 %
RB-14	Recuperar la renovación del certificado	Certificado por vencer o vencido
RB-15	Resolver una falla de DuckDNS	El dominio no resuelve a la IP elástica
RB-16	Resolver conexión o espacio de RDS	La aplicación no llega a la base o se queda sin espacio
RB-17	Crear o corregir cuentas desde la consola	Cuentas que la interfaz no permite configurar
RB-18	Apagar todo	Al terminar la calificación

RB-01 · Desplegar a producción

1. Fusionar el Pull Request. stage se despliega solo; probar ahí el cambio y que `https://<dominio>:8081/up` responda.
2. Actions → CI/CD → *Run workflow* desde `main` → *Review deployments* → *Approve*.
3. Prod toma la imagen del último despliegue sano de stage, sin construir. Verificar que `https://<dominio>/up` responda y que `docker compose ps` muestre el mismo número de imagen que stage.

RB-02 · Revertir un despliegue

- **dev o stage:** Actions → la última ejecución que terminó bien → *Re-run all jobs*.
- **prod:**

1. revertir el commit con un Pull Request (`git revert <commit>`);
2. al fusionarlo, stage queda con la versión anterior;
3. despachar prod con RB-01.

- **Las migraciones no se revierten.** Si la versión fallida cambió el esquema, se escribe otra migración que corrija. Si además dañó datos, sigue RB-03.

RB-03 · Restaurar la base a un instante concreto

Nunca se restaura sobre la instancia de producción: destruiría la evidencia e impediría comparar.

1. Ubicar en Panel → Bitácora el momento anterior al incidente. La hora del panel es de Guatemala; el comando usa UTC (sumar 6 horas).
2. Restaurar a una instancia temporal, en la misma red y con los mismos parámetros:

```
SG=$(aws ec2 describe-security-groups --filters
Name=group-name,Values=clinica-db \
  --query 'SecurityGroups[0].GroupId' --output text)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier clinica --target-db-instance-identifier
clinica-restaurada \
  --restore-time 2026-10-30T15:00:00Z \
  --db-subnet-group-name clinica --vpc-security-group-ids "$SG" \
  --db-parameter-group-name clinica-mariadb114 --no-publicly-accessible
aws rds wait db-instance-available --db-instance-identifier
clinica-restaurada
aws rds describe-db-instances --db-instance-identifier clinica-restaurada
\
  --query 'DBInstances[0].Endpoint.Address' --output text
```

3. Verificar la temporal desde la EC2, con el `.env` de prod y solo el host cambiado:

```
cd ~/clinica/prod && R=<endpoint de la temporal>
docker compose run --rm -T -e DB_HOST=$R app php artisan migrate:status |
tail -3
docker compose run --rm -T -e DB_HOST=$R app php artisan tinker --execute
\
  'foreach
(["appointments","clinical_notes","payments","psych_test_assignments","au
dit_log"] as $t) echo $t."=".DB::table($t)->count()." ";'
docker compose run --rm -T -e DB_HOST=$R app php artisan tinker --execute
\
  '$n = DB::table("clinical_notes")->orderByDesc("created_at")->first();
echo $n === null ? "sin notas" : (strlen((string)
app(App\ClinicalRecords\Infrastructure\ClinicalCipher::class)->decrypt($n
->soap_subjective)) > 0 ? "nota descifrada" : "no descifra");'
```

El tercer comando es el paso crítico: imprime si la nota se descifra, nunca su contenido.

4. Decidir con la psicóloga si se extraen datos puntuales de la temporal o si se promueve.
5. Para promoverla:
    1. renombrar la original a `clinica-anterior` y la temporal a `clinica`, con `aws rds modify-db-instance --new-db-instance-identifier … --apply-immediately`;
    2. el endpoint queda igual, así que los secretos no cambian;
    3. reiniciar los contenedores de cada entorno con `docker compose restart`;
    4. en Terraform: `terraform state rm aws_db_instance.main` y `terraform import aws_db_instance.main clinica`.
6. Anotar la causa, el instante restaurado y el RTO real en la bitácora de operación.
7. Borrar la instancia que sobra: `aws rds delete-db-instance --db-instance-identifier <instancia> --skip-final-snapshot --delete-automated-backups`.

RB-04 · Simulacro de restauración

Es RB-03 con el último instante restaurable y sin promover nada. Cuesta centavos: una `db.t4g.micro` por menos de una hora.

1. Anotar la hora de inicio y el último instante restaurable:

```
date -u +%FT%TZ
aws rds describe-db-instances --db-instance-identifier clinica \
  --query 'DBInstances[0].LatestRestorableTime' --output text
```

2. Paso 2 de RB-03 con `--target-db-instance-identifier clinica-simulacro` y `--use-latest-restorable-time` en lugar de `--restore-time`.
3. Paso 3 de RB-03, comparando los conteos con los de prod (los mismos comandos sin `-e DB_HOST`).
4. Anotar la hora de verificación y borrar la temporal (paso 7 de RB-03).
5. Llenar el registro:

Dato	Valor
Fecha	
Inicio (UTC)	
`LatestRestorableTime` al iniciar	
Temporal disponible (UTC)	
Verificado (UTC)	
RTO medido (inicio → verificado)	
RPO (inicio − `LatestRestorableTime`)	
Conteos iguales a prod	
Nota descifrada	

RB-05 · Rotar un secreto

Regla general: cambiar el valor en el Environment y repetir el último despliegue de ese entorno.

Secreto	Qué más hace falta
`DB_PASSWORD`	Nada: `deploy.sh` aplica la nueva con `ALTER USER`
`APP_KEY`	Nada; cierra las sesiones abiertas y anula los enlaces firmados pendientes
`CLINICAL_ENCRYPTION_KEY`	**No se rota solo cambiándolo:** las notas y pruebas cifradas con la anterior dejan de leerse. Primero se escribe y prueba una migración que vuelva a cifrar con la nueva
`MAIL_PASSWORD`	Crear otra contraseña de aplicación en Google y revocar la anterior
`DOCKERHUB_TOKEN`, `DUCKDNS_TOKEN`	Generar el nuevo en el proveedor y revocar el anterior
`CHATBOT_API_TOKEN`	`openssl rand -hex 32`; Laravel y el asistente lo toman en el mismo despliegue
`EC2_SSH_KEY`	Agregar la pública nueva a `~/.ssh/authorized_keys` de la EC2, cambiar `EC2_SSH_KEY` en los tres entornos y `TF_VAR_ssh_public_key` en `infra`, comprobar un despliegue y quitar la anterior
`DB_ADMIN_PASSWORD`	Subir `password_wo_version` en `rds.tf` y cambiar `TF_VAR_db_admin_password` por Pull Request; aplicar con el pipeline de infraestructura y actualizar `DB_ADMIN_PASSWORD` en los tres entornos

RB-06 · Reemplazar la instancia

1. `terraform apply -replace=aws_instance.app` en `infra`. La IP elástica pasa a la nueva: `EC2_HOST` y DuckDNS no cambian.

2. Volver a desplegar los tres entornos: *Re-run* del último Pull Request para dev, del último push a `main` para stage, y prod con RB-01. Cada despliegue clona, escribe el `.env` y emite el certificado.
3. En la máquina del operador, borrar la huella vieja: `ssh-keygen -R <ec2_host>`.

RB-07 · Diagnosticar trabajos encolados detenidos

```
docker compose ps worker scheduler
docker compose logs --tail 100 worker
docker compose exec app php artisan queue:failed
```

- Worker caído o colgado: `docker compose restart worker`.
- Trabajos fallidos por una causa ya corregida: `docker compose exec app php artisan queue:retry all`.
- La conciliación de citas corre cada 5 minutos: recupera vencimientos e impagos pendientes y cierra las sesiones terminadas. Es idempotente; para no esperar: `docker compose exec app php artisan appointments:reconcile`.

RB-08 · Diagnosticar correo no entregado

Los últimos envíos fallidos:

```
docker compose exec app php artisan tinker --execute 'echo
json_encode(DB::table("notification_dispatches")->join("notification_statuses",
"notification_statuses.id","=","notification_dispatches.status_id")->where("not
ification_statuses.code","FALLIDO")->orderByDesc("notification_dispatches.id")-
>limit(10)->get(["template_code","attempts","last_error"]));'
```

Error	Causa	Qué hacer
`535 … Username and Password not accepted`	Se revocó la contraseña de aplicación	RB-05, `MAIL_PASSWORD`
`550 5.4.5 Daily sending quota exceeded`	Límite diario de Gmail, unos 500 mensajes	Esperar 24 horas; si se repite, pasar a un servicio transaccional
Ninguno, pero no llega	Filtro de spam del destinatario	Revisar la carpeta de spam; el remitente es siempre la cuenta de Gmail
Nada sale	Worker detenido	RB-07

Los avisos a pacientes esperan la ventana de silencio (21:00 a 07:00 de Guatemala). Los códigos de segundo factor, los enlaces de recuperación y las alertas urgentes salen siempre.

RB-09 · Recuperar un entorno degradado sin desplegar

1. `docker compose ps` y `docker compose logs --tail 100 app web` del entorno.
2. Reiniciar el entorno: `docker compose restart`.
3. Si es el host: `free -h`, `df -h` y `docker system df`. Un `sudo reboot` recupera los tres entornos, que arrancan solos.
4. Si nada basta, RB-06.

RB-10 · Actualizar lo que sabe el asistente

El asistente consulta la clínica por la API, no un archivo. No hace falta desplegar:

- preguntas frecuentes: Panel → Preguntas frecuentes (también se ven en la landing);
- servicios y tarifas: Panel → Servicios y tarifas;
- vacaciones y ausencias: Panel → Bloqueos de agenda.

RB-11 · Responder a una alerta de riesgo

Las dos alertas son urgentes y salen aun en horas de silencio.

NT-24, señal de crisis en el asistente. El widget ya mostró la línea de crisis y cerró la conversación. El asistente no guarda mensajes ni datos de la persona. Cada mes se revisan las alertas: un falso negativo se corrige de inmediato con un caso nuevo en la batería de crisis.

NT-25, ítem 9 del PHQ-9 mayor que cero.

1. Abrir el enlace del correo: el resultado está en el panel y la lectura queda en la bitácora.
2. Contactar al paciente el mismo día, al teléfono de su ficha.
3. Documentar el contacto en la nota SOAP de la siguiente sesión.

RB-12 · Responder a un incidente de seguridad

1. **Contener:** rotar lo expuesto (RB-05). Si la EC2 está comprometida, quitar su llave de `authorized_keys`, tomar una instantánea del disco para el análisis y reemplazarla (RB-06).
2. **Evaluar:** la bitácora del panel muestra quién leyó cada expediente y cada intento denegado; Actions, quién desplegó qué; CloudTrail, 90 días de llamadas a AWS.
3. **Avisar** a la psicóloga con lo que se sabe y lo que se hizo.
4. **Restaurar** si hubo datos alterados (RB-03).
5. **Aprender:** post-mortem y acciones en la bitácora.

RB-13 · Liberar disco en la EC2

1. Medir: `df -h /` y `docker system df`.
2. Borrar imágenes y capas sin uso: `docker image prune -af` y `docker builder prune -af`. Los contenedores en marcha no se tocan.
3. Si persiste, buscar el origen: `sudo du -xh / --max-depth=2 | sort -h | tail -15`. Los registros de contenedor están acotados; revisar `~/clinica/tls.log`.
4. Si el disco de 30 GB no alcanza, subir `volume_size` en `ec2.tf` por Pull Request y aplicar con el pipeline de infraestructura; luego `sudo growpart /dev/nvme0n1 1 && sudo resize2fs /dev/nvme0n1p1`.

RB-14 · Recuperar la renovación del certificado

El cron renueva cada día a las 04:17 con lego y el DNS de DuckDNS. Let's Encrypt emite por 90 días y lego renueva en los últimos 30.

1. Leer el error: `tail -n 50 ~/clinica/tls.log`.
2. Si es de DuckDNS (`KO` o sin respuesta), seguir RB-15 y volver.
3. Renovar a mano: `flock ~/clinica/tls.lock ~/clinica/tls.sh`. Al terminar recarga nginx en los tres entornos.

4. Comprobar la fecha: `openssl s_client -connect <dominio>:443 -servername <dominio> </dev/null | openssl x509 -noout -dates`.
5. Si Let's Encrypt limita las emisiones, esperar la ventana que indique el error. El certificado anterior sigue sirviendo hasta su vencimiento.

RB-15 · Resolver una falla de DuckDNS

1. `dig +short <dominio>` debe dar la IP elástica (`terraform output ec2_host`).
2. Si la IP es otra, actualizarla con el token, desde la EC2: `curl "https://www.duckdns.org/update?domains=<subdominio>&token=<token>&ip=<ec2_host>"`.
3. Si DuckDNS está caído, la aplicación sigue viva: el personal puede entrar temporalmente por `https://<ec2_host>` aceptando la advertencia del certificado, solo para urgencias.
4. Si la caída dura más de un día, registrar un dominio propio y apuntarlo a la IP elástica. Ese cambio incluye `DOMAIN` y el certificado.

RB-16 · Resolver conexión o espacio de RDS

1. Estado: `aws rds describe-db-instances --db-instance-identifier clinica --query 'DBInstances[0].[DBInstanceStatus,AllocatedStorage]'`.
2. Conexión desde la EC2: `docker compose exec app php artisan migrate:status | tail -2`. Un error 1045 indica la contraseña (RB-05); un *timeout*, el grupo `clinica-db` o el estado de la instancia.
3. Si está en `storage-full`, aumentar `allocated_storage` en `rds.tf` y aplicar. RDS crece sin detener la base.
4. Si la instancia falló, restaurar con RB-03 y el último instante restaurable.

RB-17 · Crear o corregir cuentas desde la consola

La interfaz exige contraseñas de 12 caracteres no filtradas y marca como temporal toda contraseña que pone el personal. Para cuentas de demostración o correcciones

puntuales se usa la consola. Nunca se crea una segunda psicóloga: el sistema supone una sola fila en `therapists`.

```
cd ~/clinica/prod
docker compose exec -T app php artisan tinker --execute "$(cat cuentas.php)"
use App\Models\{Patient, User};
$rol = fn (string $c) => DB::table('roles')->where('code', $c)->value('id');
$acceso = ['password' => '<contraseña>', 'must_change_password' => false,
'temp_password_expires_at' => null, 'is_active' => true];
User::findOrFail(DB::table('therapists')->value('id'))->forceFill(['email' =>
'<correo de la psicóloga>'] + $acceso)->save();
User::updateOrCreate(['email' => '<correo de recepción>'], ['role_id' =>
$rol('secretary'), 'name' => 'Recepción'] + $acceso);
$p = User::updateOrCreate(['email' => '<correo del paciente>'], ['role_id' =>
$rol('patient'), 'name' => '<nombre>'] + $acceso);
Patient::firstOrCreate(['id' => $p->getKey()], ['nit' => 'CF']);
```

Antes de escribir, consultar quién tiene ya esos correos: un correo es único. Después, releer las cuentas y actualizar el secreto `ADMIN_EMAIL`, para que una base recreada siembre a la misma psicóloga. Ningún correo ni contraseña real va al repositorio, que es público.

RB-18 · Apagar todo

1. Quitar la protección contra borrado de RDS: `aws rds modify-db-instance --db-instance-identifier clinica --no-deletion-protection --apply-immediately`.
2. Vaciar los tres buckets de archivos, con sus versiones.
3. Borrar los puntos de recuperación de la bóveda `clinica` de AWS Backup.
4. `terraform destroy` en `infra`. Queda la instantánea final `clinica-final`.
5. `terraform destroy` en `infra/bootstrap`, tras vaciar el bucket del estado.
6. Borrar la llave de `clinica-admin` y la contraseña de aplicación de Gmail.

Bitácora de operación

Fecha	Procedimiento	Resultado
2026-10-25	RB-01	Primer despliegue a prod: la imagen 11 de stage, sin construir (ejecución 13 de CI/CD, aprobada). `https://pf-clinica.duckdns.org/up` respondió a las 09:18 UTC
2026-10-25	RB-17	Cuentas de demostración en prod: la psicóloga con su correo real, y una cuenta de recepción y otra de paciente. Relectura correcta de rol, contraseña y ficha